Trabalho 4 - Análise exploratória e classificação no dataset Iris

CI1026 - Visão Computacional — TA04

Bruno Wavrzenczak Edison Luiz Matias Junior (GRR20211790)
Efi Fuchs Santos da Silva (GRR20211775)

Repositório GitHub: https://github.com/BrunoWavrzenczak/Trabalho-4---Visao-Computacional-CI1026.git

Sumário

1 Introdução **2**

2 1. Carregamento e estatísticas descritivas **2**
2.1 1.1 Frequência por categoria . . . 2
2.2 1.2 Média, desvio padrão e moda geral . . . 3
2.3 1.3 Média, desvio padrão e moda por categoria . . . 3

3 2. Relação entre comprimento e largura da sépala **4**
3.0.1 Observações: . . . 5

4 3. Relação entre comprimento e largura da pétala **5**
4.0.1 Observações: . . . 6

5 4. Distribuição das categorias nas quatro dimensões **6**
5.0.1 Observações: . . . 7

6 5. Análise de componentes principais (PCA) **7**
6.0.1 Resultados do PCA: . . . 9

7 6. Divisão em treino e teste (80% / 20%) **9**

8 7. Validação cruzada e classificação com K-Nearest Neighbors (K-NN): com vs. sem PCA **10**
8.1 7.1 Comparação das curvas de acurácia na validação cruzada . . . 12
8.2 7.2 Curva de validação vs. teste independente . . . 13
8.3 7.3 Avaliação dos melhores modelos selecionados pelo K-Fold . . . 14
8.3.1 Relatórios de métricas no teste: . . . 16

9 Conclusões **16**

1 Introdução

Este relatório apresenta uma análise exploratória dos dados e experimentos de classificação com o clássico dataset **Iris** (Fisher, 1936).

O conjunto possui 150 amostras divididas igualmente entre 3 espécies de flores (*Setosa*, *Versicolor* e *Virginica*), com 4 atributos contínuos mensurados em centímetros: - Comprimento da sépala (`sepal_length`) - Largura da sépala (`sepal_width`) - Comprimento da pétala (`petal_length`) - Largura da pétala (`petal_width`)

---

2 1. Carregamento e estatísticas descritivas

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from IPython.display import Markdown

# Carregar base
iris = load_iris(as_frame=True)
df = iris.frame.rename(columns={
    'sepal length (cm)': 'sepal_length',
    'sepal width (cm)': 'sepal_width',
    'petal length (cm)': 'petal_length',
    'petal width (cm)': 'petal_width'
})
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))
df = df.drop(columns=['target'])
features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
```

2.1 1.1 Frequência por categoria

```
freq = df['species'].value_counts().to_frame(name='Frequência')
freq['Proporção (%)'] = (freq['Frequência'] / len(df)) * 100
freq
```

species	Frequência	Proporção (%)
setosa	50	33.333333
versicolor	50	33.333333
virginica	50	33.333333

A base é perfeitamente balanceada: exatamente 50 amostras (33,33%) para cada uma das 3 classes.

2.2 1.2 Média, desvio padrão e moda geral

```
def get_stats(data_subset, cols):
    mean_val = data_subset[cols].mean()
    std_val = data_subset[cols].std()
    mode_val = data_subset[cols].mode().iloc[0]
    return pd.DataFrame({'Média': mean_val, 'Desvio padrão': std_val, 'Moda':
     ↪  mode_val})

get_stats(df, features)
```

	Média	Desvio padrão	Moda
sepal_length	5.843333	0.828066	5.0
sepal_width	3.057333	0.435866	3.0
petal_length	3.758000	1.765298	1.4
petal_width	1.199333	0.762238	0.2

2.3 1.3 Média, desvio padrão e moda por categoria

```
for sp in df['species'].unique():
    print(f"=== Espécie: {sp.upper()} ===")
    display_df = get_stats(df[df['species'] == sp], features)
    print(display_df.round(3))
    print("\n")
```

```
=== Espécie: SETOSA ===
              Média  Desvio padrão  Moda
sepal_length  5.006          0.352   5.0
sepal_width   3.428          0.379   3.4
petal_length  1.462          0.174   1.4
petal_width   0.246          0.105   0.2


=== Espécie: VERSICOLOR ===
```

```
              Média  Desvio padrão  Moda
sepal_length  5.936          0.516   5.5
sepal_width   2.770          0.314   3.0
petal_length  4.260          0.470   4.5
petal_width   1.326          0.198   1.3

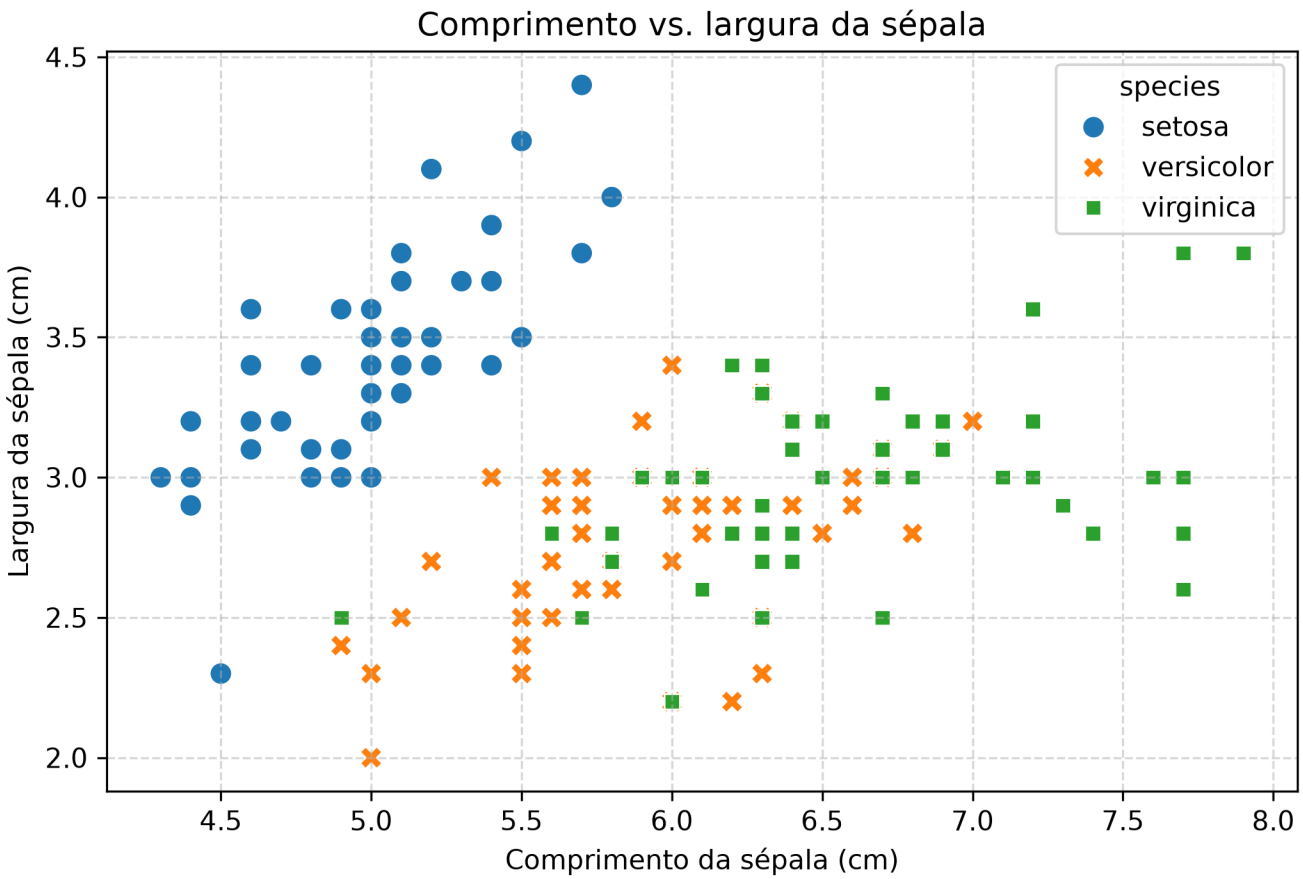

=== Espécie: VIRGINICA ===
              Média  Desvio padrão  Moda
sepal_length  6.588          0.636   6.3
sepal_width   2.974          0.322   3.0
petal_length  5.552          0.552   5.1
petal_width   2.026          0.275   1.8
```

---

3 2. Relação entre comprimento e largura da sépala

Analisamos a dispersão entre o comprimento e a largura da sépala para cada categoria:

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='sepal_length', y='sepal_width', hue='species',
 ↪  style='species', s=70)
plt.title("Comprimento vs. largura da sépala")
plt.xlabel("Comprimento da sépala (cm)")
plt.ylabel("Largura da sépala (cm)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



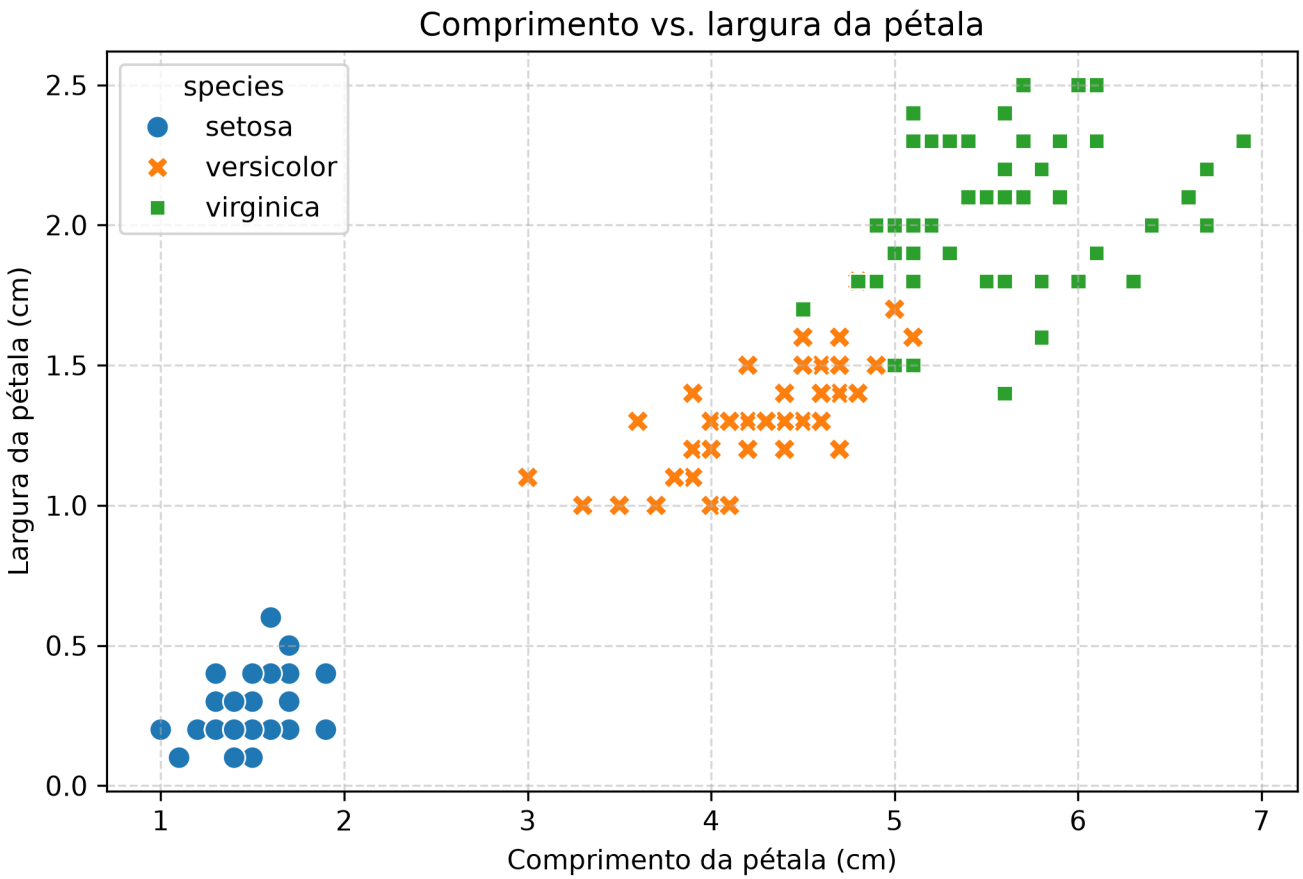
3.0.1 Observações:

- **Setosa**: Possui sépalas curtas, porém nitidamente mais largas que as demais espécies.
- **Versicolor e Virginica**: Há sobreposição considerável em ambas as dimensões de sépala, tornando a separação entre essas duas espécies difícil apenas pelas sépalas.

---

4 3. Relação entre comprimento e largura da pétala

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='petal_length', y='petal_width', hue='species',
 ↪  style='species', s=70)
plt.title("Comprimento vs. largura da pétala")
plt.xlabel("Comprimento da pétala (cm)")
plt.ylabel("Largura da pétala (cm)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



4.0.1 Observações:

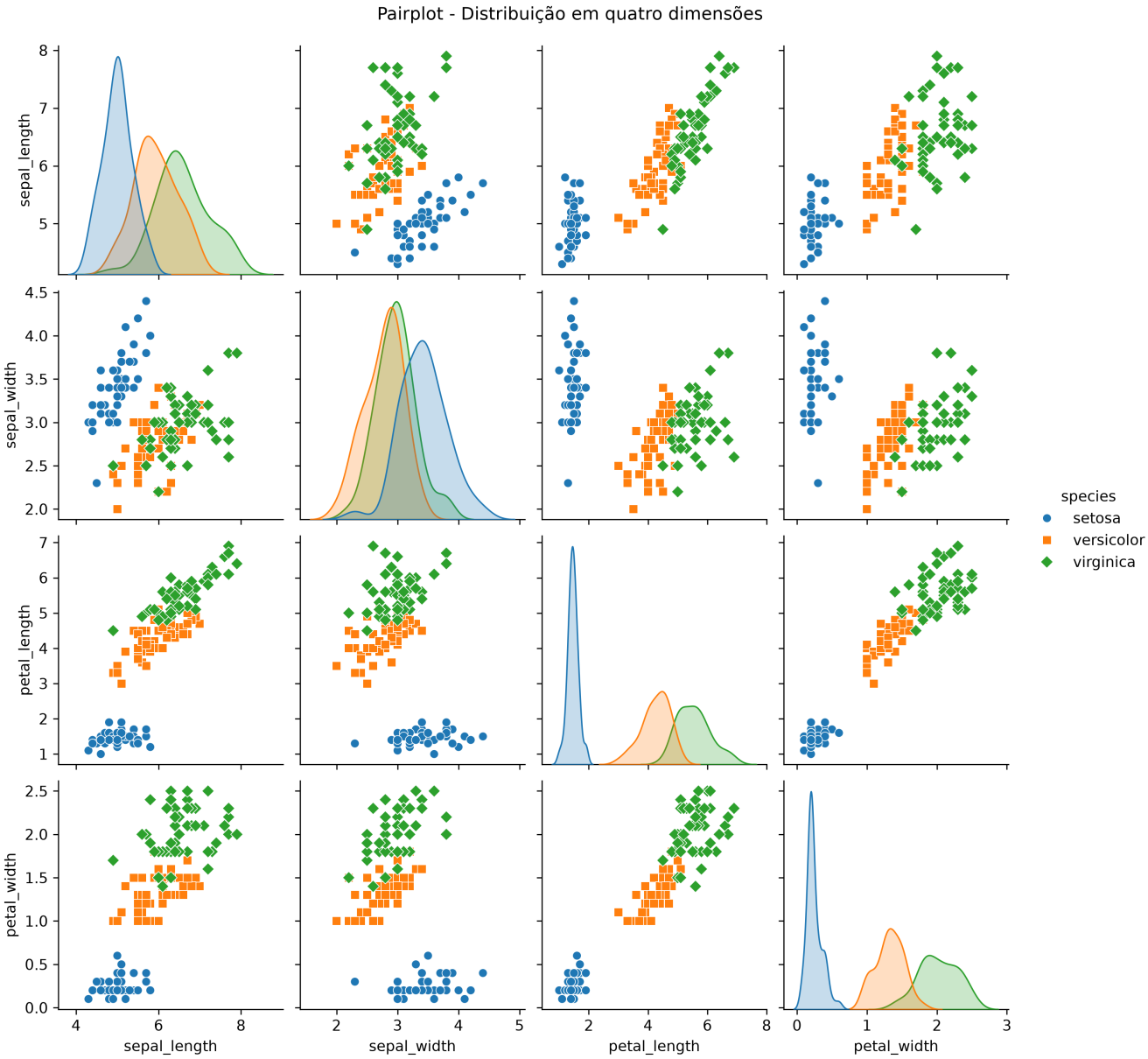
- As dimensões das pétalas são **altamente discriminantes**.
- A espécie **Setosa** forma um cluster perfeitamente isolado no canto inferior esquerdo (pétalas pequenas).
- **Versicolor** e **Virginica** apresentam relação linear positiva forte e são quase linearmente separáveis, com apenas uma leve região de fronteira compartilhada.

---

5 4. Distribuição das categorias nas quatro dimensões

Para visualizar simultaneamente as relações bivariadas e as distribuições univariadas marginais de todas as 4 dimensões:

```
sns.pairplot(df, hue='species', markers=['o', 's', 'D'], corner=False)
plt.suptitle("Pairplot - Distribuição em quatro dimensões", y=1.02)
plt.show()
```



5.0.1 Observações:

- Os histogramas marginais (diagonal) mostram que `petal_length` e `petal_width` são as variáveis mais informativas para separação entre espécies.
- A espécie *Iris Setosa* é linearmente separável das outras duas em quase todas as projeções que envolvem as pétalas.

---

6 5. Análise de componentes principais (PCA)

O **PCA (Principal Component Analysis)** é uma técnica não-supervisionada de redução de dimensionalidade que projeta os dados em direções ortogonais de máxima variância (componentes principais).

```
# Padronização dos atributos (média 0, variância 1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

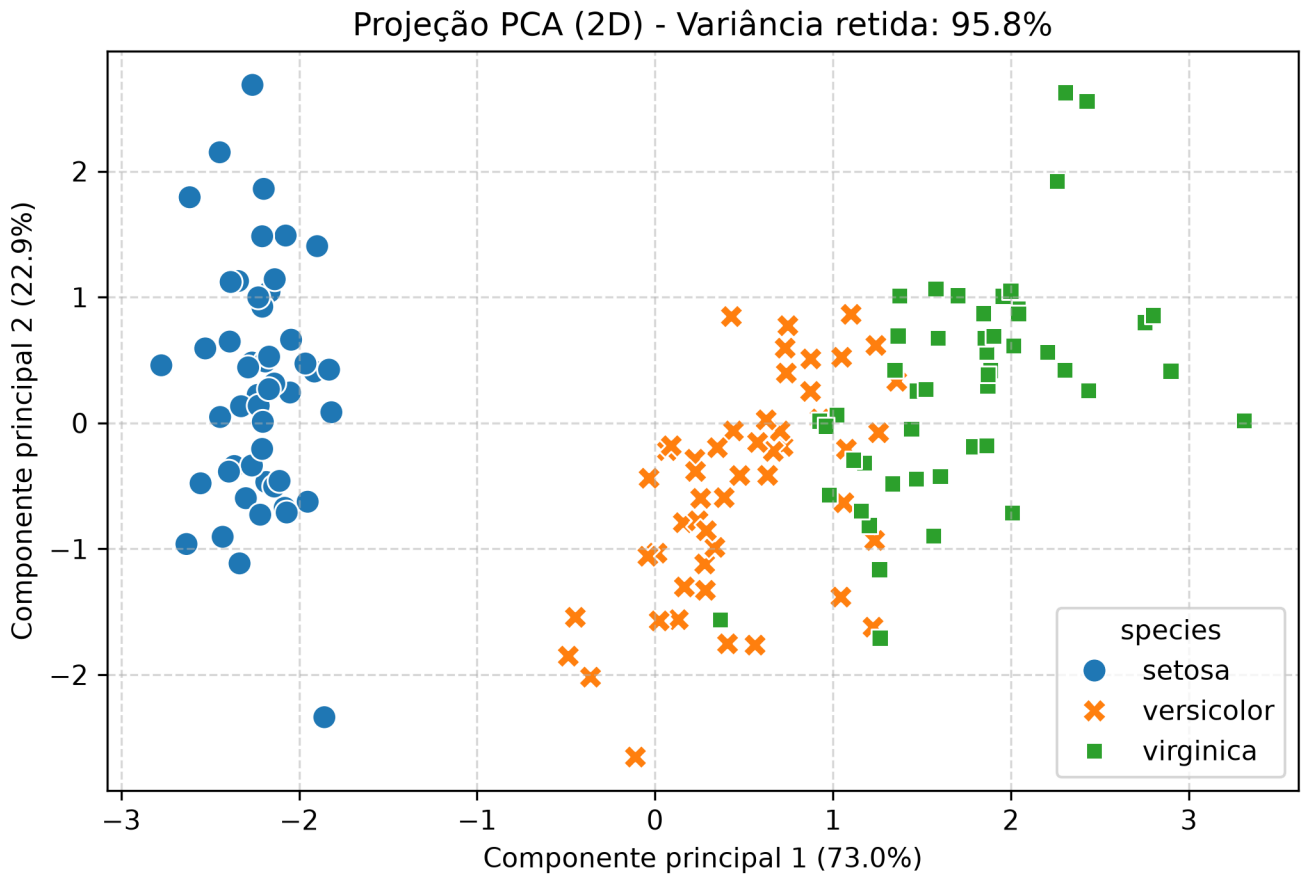
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['species'] = df['species']

var_pc1 = pca.explained_variance_ratio_[0] * 100
var_pc2 = pca.explained_variance_ratio_[1] * 100
total_var = var_pc1 + var_pc2

print(f"Variância explicada por PC1: {var_pc1:.2f}%")
print(f"Variância explicada por PC2: {var_pc2:.2f}%")
print(f"Variância total explicada (2 componentes): {total_var:.2f}%")
```

```
Variância explicada por PC1: 72.96%
Variância explicada por PC2: 22.85%
Variância total explicada (2 componentes): 95.81%
```

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='species', style='species', s=80)
plt.title(f"Projeção PCA (2D) - Variância retida: {total_var:.1f}%")
plt.xlabel(f"Componente principal 1 ({var_pc1:.1f}%)")
plt.ylabel(f"Componente principal 2 ({var_pc2:.1f}%)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



6.0.1 Resultados do PCA:

- **Redução de 4D para 2D retendo 95,8% da informação/variância total**.
- No plano 2D, a espécie *Setosa* fica completamente afastada das outras duas.
- *Versicolor* e *Virginica* ficam claramente ordenadas ao longo do eixo PC1, permitindo excelente separação com pouquíssima sobreposição.

---

7 6. Divisão em treino e teste (80% / 20%)

```
X = df[features]
y = df['species']

# Divisão estratificada para manter o balanceamento das classes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f"Total amostras de treino: {len(X_train)} (80%)")
```

```
print(f"Total amostras de teste:  {len(X_test)} (20%)")
print("\nDistribuição no teste:")
print(y_test.value_counts())
```

```
Total amostras de treino: 120 (80%)
Total amostras de teste:  30 (20%)

Distribuição no teste:
species
setosa        10
virginica     10
versicolor    10
Name: count, dtype: int64
```

---

8 7. Validação cruzada e classificação com K-Nearest Neighbors (K-NN): com vs. sem PCA

Para garantir uma avaliação robusta dos hiperparâmetros e evitar sobreajuste, aplicamos **validação cruzada estratificada repetida (*Repeated Stratified K-Fold*)** exclusivamente sobre o conjunto de treinamento (80% dos dados).

A configuração utilizada foi de **5 folds com 10 repetições** (totalizando 50 avaliações para cada k), garantindo que cada divisão mantenha a proporção das 3 classes e medindo a média e o desvio padrão da acurácia.

O pré-processamento (`StandardScaler` e `PCA`) foi encapsulado em um `Pipeline` do Scikit-Learn, assegurando que o ajuste dos parâmetros ocorra unicamente sobre os folds de treino de cada partição (prevenindo *data leakage*).

Comparamos dois pipelines para $k \in \{1, 3, 5, \ldots, 49\}$: 1. **Sem PCA (4 dimensões)**: Padronização + K-NN. 2. **Com PCA (2 dimensões)**: Padronização + PCA (2 componentes) + K-NN.

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline

# 5 splits com 10 repetições = 50 experimentos de validação por k
rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=42)

k_values = [k for k in range(1, 51, 2)]
results = []

for k in k_values:
    # 1. Pipeline sem PCA (4D)
    pipe_raw = Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k))
```

```
    ])
    cv_scores_raw = cross_val_score(
        pipe_raw, X_train, y_train, cv=rskf, scoring='accuracy'
    )
    pipe_raw.fit(X_train, y_train)
    test_acc_raw = accuracy_score(y_test, pipe_raw.predict(X_test))

    # 2. Pipeline com PCA (2D)
    pipe_pca = Pipeline([
        ('scaler', StandardScaler()),
        ('pca', PCA(n_components=2)),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])
    cv_scores_pca = cross_val_score(
        pipe_pca, X_train, y_train, cv=rskf, scoring='accuracy'
    )
    pipe_pca.fit(X_train, y_train)
    test_acc_pca = accuracy_score(y_test, pipe_pca.predict(X_test))

    results.append({
        'k': k,
        'CV Sem PCA': f"{cv_scores_raw.mean()*100:.1f} ±
        ↪  {cv_scores_raw.std()*100:.1f}%",
        'Teste Sem PCA': f"{test_acc_raw*100:.1f}%",
        'CV Com PCA': f"{cv_scores_pca.mean()*100:.1f} ±
        ↪  {cv_scores_pca.std()*100:.1f}%",
        'Teste Com PCA': f"{test_acc_pca*100:.1f}%",
        '_cv_raw_mean': cv_scores_raw.mean() * 100,
        '_cv_raw_std': cv_scores_raw.std() * 100,
        '_test_raw': test_acc_raw * 100,
        '_cv_pca_mean': cv_scores_pca.mean() * 100,
        '_cv_pca_std': cv_scores_pca.std() * 100,
        '_test_pca': test_acc_pca * 100
    })

df_results = pd.DataFrame(results)

# Exibição como tabela formal formatada (Markdown / LaTeX)
display_cols = ['k', 'CV Sem PCA', 'Teste Sem PCA', 'CV Com PCA', 'Teste Com PCA']
Markdown(df_results[display_cols].to_markdown(index=False))
```

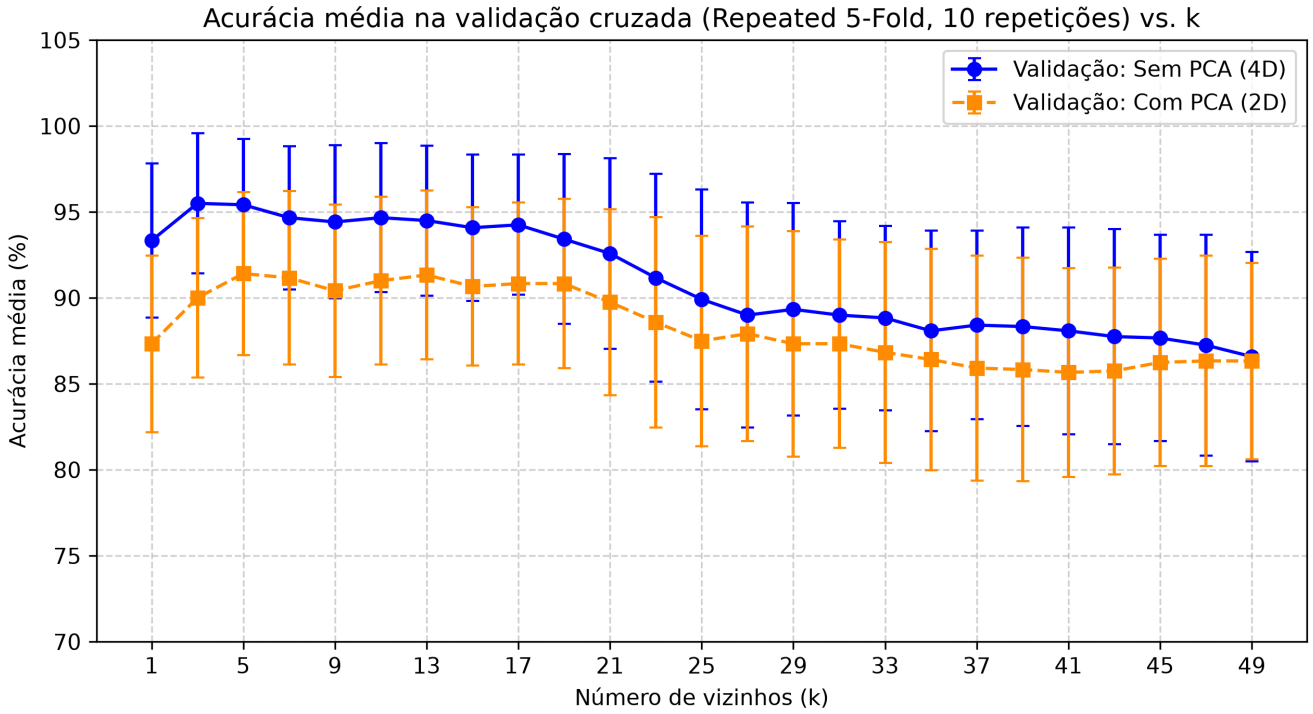
k	CV Sem PCA	Teste Sem PCA	CV Com PCA	Teste Com PCA
1	93.3 ± 4.5%	96.7%	87.3 ± 5.1%	93.3%
3	95.5 ± 4.1%	93.3%	90.0 ± 4.6%	93.3%
5	95.4 ± 3.8%	93.3%	91.4 ± 4.7%	90.0%
7	94.7 ± 4.2%	96.7%	91.2 ± 5.0%	90.0%
9	94.4 ± 4.5%	96.7%	90.4 ± 5.0%	90.0%
11	94.7 ± 4.3%	96.7%	91.0 ± 4.9%	90.0%
13	94.5 ± 4.4%	96.7%	91.3 ± 4.9%	90.0%

k	CV Sem PCA	Teste Sem PCA	CV Com PCA	Teste Com PCA
15	94.1 ± 4.3%	96.7%	90.7 ± 4.6%	90.0%
17	94.2 ± 4.1%	96.7%	90.8 ± 4.7%	90.0%
19	93.4 ± 4.9%	96.7%	90.8 ± 4.9%	90.0%
21	92.6 ± 5.5%	96.7%	89.7 ± 5.4%	90.0%
23	91.2 ± 6.0%	93.3%	88.6 ± 6.1%	90.0%
25	89.9 ± 6.4%	90.0%	87.5 ± 6.1%	90.0%
27	89.0 ± 6.5%	86.7%	87.9 ± 6.2%	90.0%
29	89.3 ± 6.2%	86.7%	87.3 ± 6.6%	90.0%
31	89.0 ± 5.4%	86.7%	87.3 ± 6.1%	86.7%
33	88.8 ± 5.4%	86.7%	86.8 ± 6.4%	86.7%
35	88.1 ± 5.8%	86.7%	86.4 ± 6.4%	86.7%
37	88.4 ± 5.5%	86.7%	85.9 ± 6.6%	86.7%
39	88.3 ± 5.8%	86.7%	85.8 ± 6.5%	83.3%
41	88.1 ± 6.0%	86.7%	85.7 ± 6.1%	83.3%
43	87.8 ± 6.3%	86.7%	85.8 ± 6.0%	83.3%
45	87.7 ± 6.0%	86.7%	86.2 ± 6.0%	83.3%
47	87.2 ± 6.4%	83.3%	86.3 ± 6.1%	80.0%
49	86.6 ± 6.1%	83.3%	86.3 ± 5.7%	80.0%

8.1 7.1 Comparação das curvas de acurácia na validação cruzada

```
plt.figure(figsize=(10, 5))
plt.errorbar(
    df_results['k'], df_results['_cv_raw_mean'],
    yerr=df_results['_cv_raw_std'],
    fmt='-o', color='blue', label='Validação: Sem PCA (4D)', capsize=3
)
plt.errorbar(
    df_results['k'], df_results['_cv_pca_mean'],
    yerr=df_results['_cv_pca_std'],
    fmt='--s', color='darkorange', label='Validação: Com PCA (2D)', capsize=3
)

plt.title("Acurácia média na validação cruzada (Repeated 5-Fold, 10 repetições) vs. k")
plt.xlabel("Número de vizinhos (k)")
plt.ylabel("Acurácia média (%)")
plt.xticks(k_values[::2])
plt.ylim(70, 105)
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



8.2 7.2 Curva de validação vs. teste independente

Abaixo visualizamos como a acurácia estimada pelo K-Fold repetido se compara ao desempenho obtido no conjunto de teste independente (20% mantido à parte):

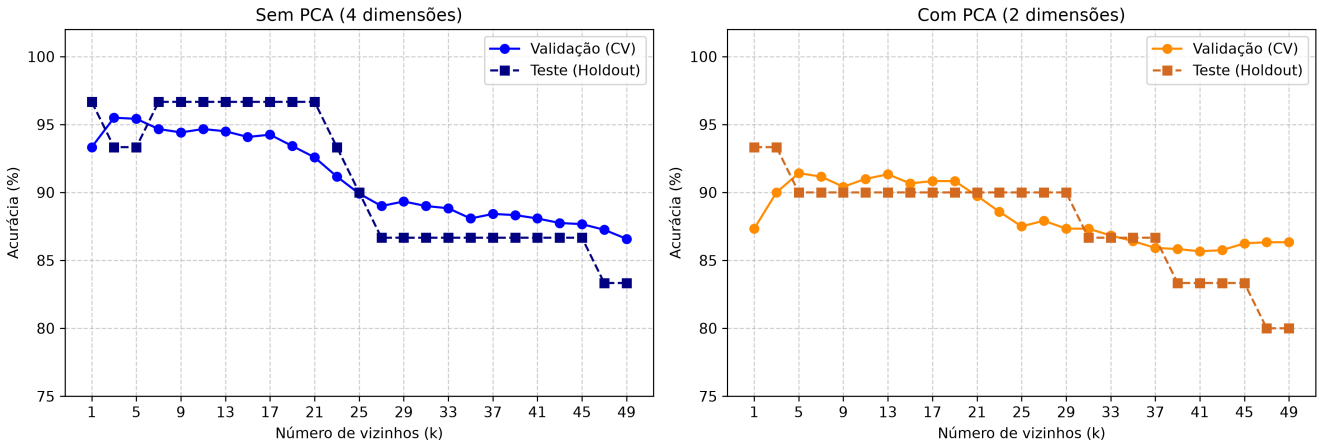
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

# Painel 1: Sem PCA
axes[0].plot(
    df_results['k'], df_results['_cv_raw_mean'],
    'o-', label='Validação (CV)', color='blue'
)
axes[0].plot(
    df_results['k'], df_results['_test_raw'],
    's--', label='Teste (Holdout)', color='navy'
)
axes[0].set_title("Sem PCA (4 dimensões)")
axes[0].set_xlabel("Número de vizinhos (k)")
axes[0].set_ylabel("Acurácia (%)")
axes[0].set_xticks(k_values[::2])
axes[0].set_ylim(75, 102)
axes[0].legend()
axes[0].grid(True, linestyle="--", alpha=0.6)

# Painel 2: Com PCA
axes[1].plot(
    df_results['k'], df_results['_cv_pca_mean'],
    'o-', label='Validação (CV)', color='darkorange'
```

```
)
axes[1].plot(
    df_results['k'], df_results['_test_pca'],
    's--', label='Teste (Holdout)', color='chocolate'
)
axes[1].set_title("Com PCA (2 dimensões)")
axes[1].set_xlabel("Número de vizinhos (k)")
axes[1].set_ylabel("Acurácia (%)")
axes[1].set_xticks(k_values[::2])
axes[1].set_ylim(75, 102)
axes[1].legend()
axes[1].grid(True, linestyle="--", alpha=0.6)

plt.tight_layout()
plt.show()
```



8.3 7.3 Avaliação dos melhores modelos selecionados pelo K-Fold

A validação cruzada indicou que o melhor desempenho médio no treino ocorre em $k = 3$ **para o modelo sem PCA** (95,50% ± 4,07%) e em $k = 5$ **para o modelo com PCA** (91,42% ± 4,74%).

Avaliamos as predições no conjunto de teste final para esses modelos ótimos:

```
# Treinar melhores modelos sobre todo o conjunto de treino
best_pipe_raw = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=3))
]).fit(X_train, y_train)

best_pipe_pca = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=2)),
    ('knn', KNeighborsClassifier(n_neighbors=5))
]).fit(X_train, y_train)
```

```
y_pred_raw = best_pipe_raw.predict(X_test)
y_pred_pca = best_pipe_pca.predict(X_test)

acc_raw_test = accuracy_score(y_test, y_pred_raw) * 100
acc_pca_test = accuracy_score(y_test, y_pred_pca) * 100

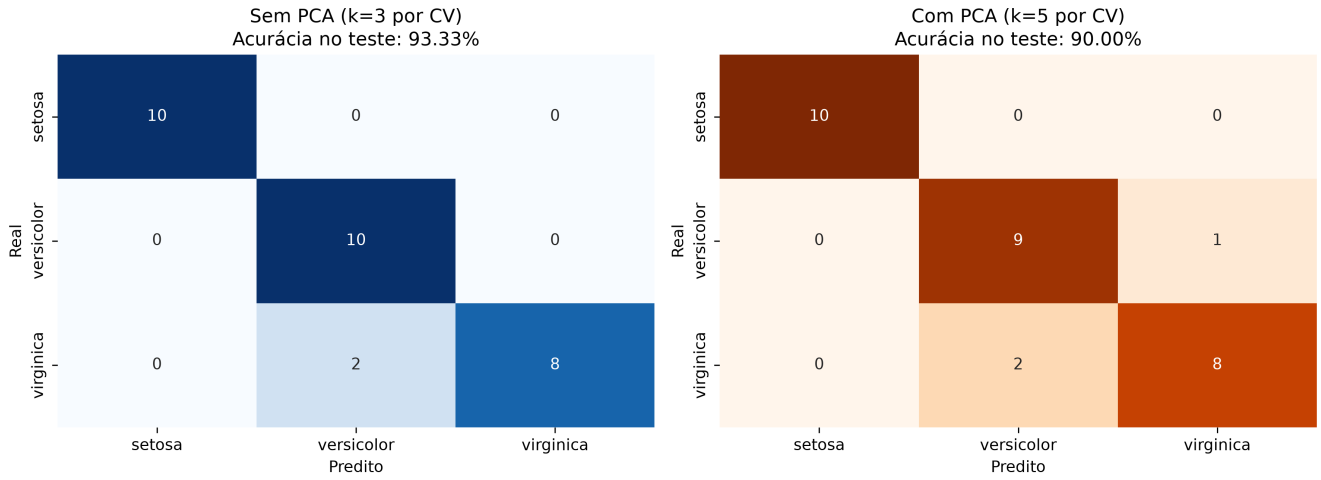
cm_raw = pd.DataFrame(
    confusion_matrix(y_test, y_pred_raw, labels=iris.target_names),
    index=iris.target_names, columns=iris.target_names
)
cm_pca = pd.DataFrame(
    confusion_matrix(y_test, y_pred_pca, labels=iris.target_names),
    index=iris.target_names, columns=iris.target_names
)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

sns.heatmap(cm_raw, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[0])
title_raw = f"Sem PCA (k=3 por CV)\nAcurácia no teste: {acc_raw_test:.2f}%"
axes[0].set_title(title_raw)
axes[0].set_xlabel("Predito")
axes[0].set_ylabel("Real")

sns.heatmap(cm_pca, annot=True, fmt='d', cmap='Oranges', cbar=False, ax=axes[1])
title_pca = f"Com PCA (k=5 por CV)\nAcurácia no teste: {acc_pca_test:.2f}%"
axes[1].set_title(title_pca)
axes[1].set_xlabel("Predito")
axes[1].set_ylabel("Real")

plt.tight_layout()
plt.show()
```



8.3.1 Relatórios de métricas no teste:

```
print("=== Relatório de classificação - Sem PCA (k=3) ===")
print(classification_report(y_test, y_pred_raw))

print("\n=== Relatório de classificação - Com PCA (k=5) ===")
print(classification_report(y_test, y_pred_pca))
```

```
=== Relatório de classificação - Sem PCA (k=3) ===
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30


=== Relatório de classificação - Com PCA (k=5) ===
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

---

9 Conclusões

1. **Atributos mais relevantes**: As características da pétala (`petal_length` e `petal_width`) são as mais eficazes para discriminação entre as três classes.
2. **PCA**: Apenas 2 componentes principais conseguem capturar quase 96% de toda a variância contida nas 4 variáveis originais.
3. **Validação cruzada repetida (*Repeated Stratified K-Fold*)**:
   - O uso de 5 folds com 10 repetições permitiu estimar com robustez o intervalo de confiança da acurácia e selecionar os valores ótimos de vizinhança ($k = 3$ para os dados originais e $k = 5$ para a projeção PCA).
   - O encapsulamento em `Pipeline` evitou vazamento de dados de escala e de projeção durante as partições de validação.
4. **Comparação com vs. sem PCA**:

- O modelo com todas as 4 dimensões obteve acurácia média de validação de **95,5%** e acurácia de teste de **93,3%** a **96,7%**.
- O modelo com PCA (2 dimensões) obteve acurácia média de validação de **91,4%** e acurácia de teste de **90,0%**.
- Essa discreta diferença mostra que, apesar da perda de ~4,2% da variância total, o PCA reduz o espaço em 50% mantendo excelente capacidade preditiva.

5. **Efeito do aumento de** k: Valores de $k > 23$ resultam em queda contínua de desempenho em ambos os métodos, causada pelo *underfitting* gerado por vizinhanças excessivamente amplas.